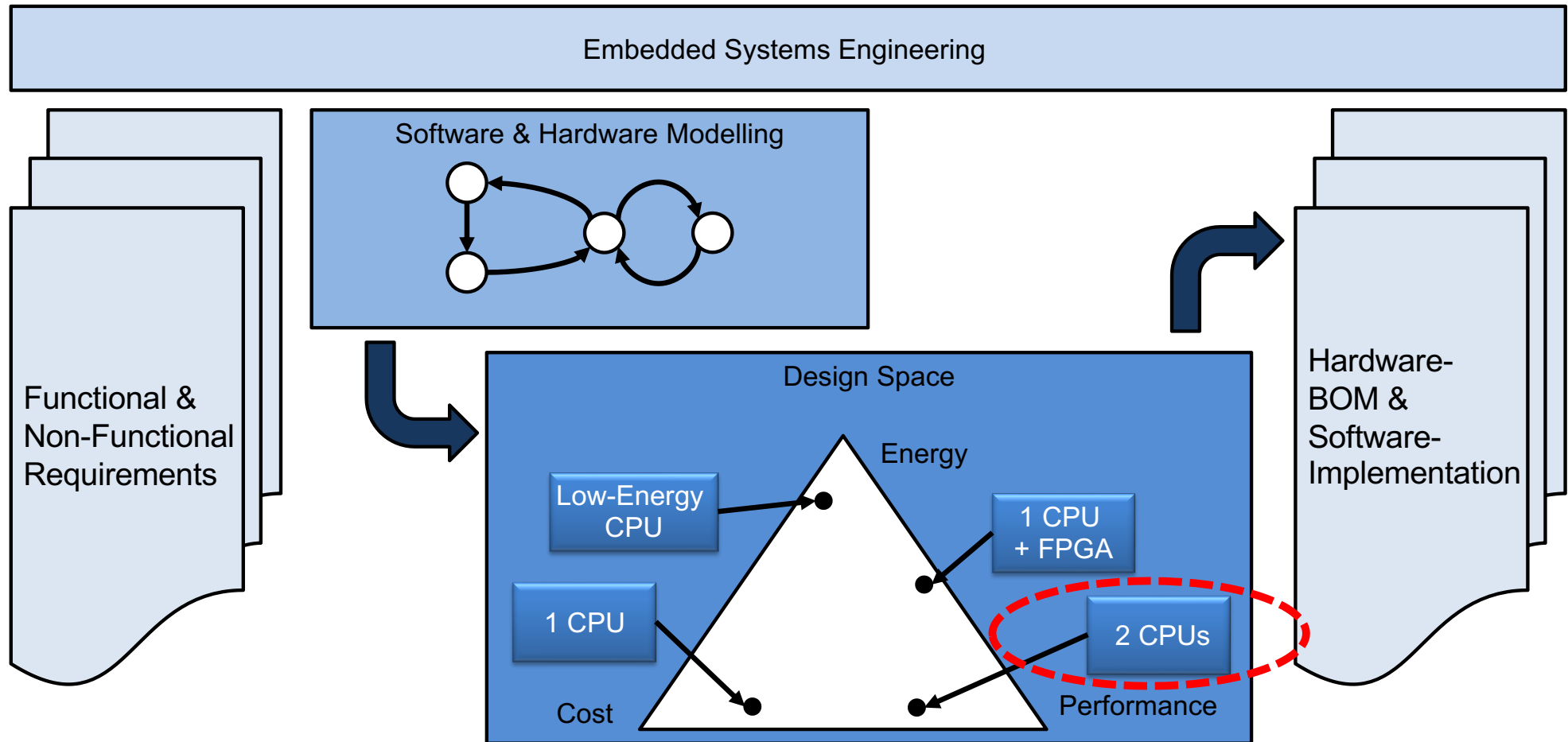


Embedded Software Engineering

Uniform Memory Access und Multi-Prozessor-Systeme

Simon Künzli, Tobias Welti
HS 2025

Advance Organizer ESE



Goals

After today's lesson, you should be able to ...

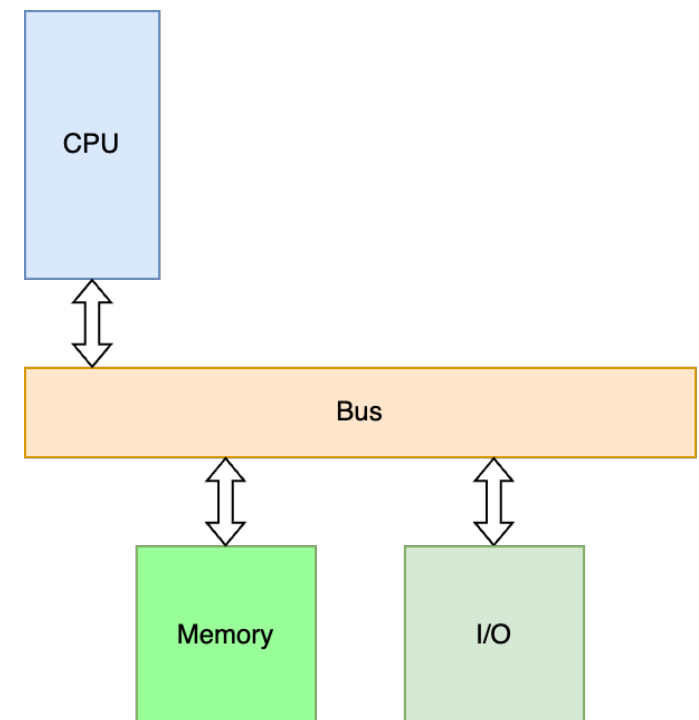
- ... explain traditional CPU architecture and its performance limitations.
- ... explain uniform and non-uniform memory access and their advantages/limitations
- ... discuss the advantages of big.LITTLE multiprocessor systems

Traditional Memory Access

Recap CT

Single CPU system

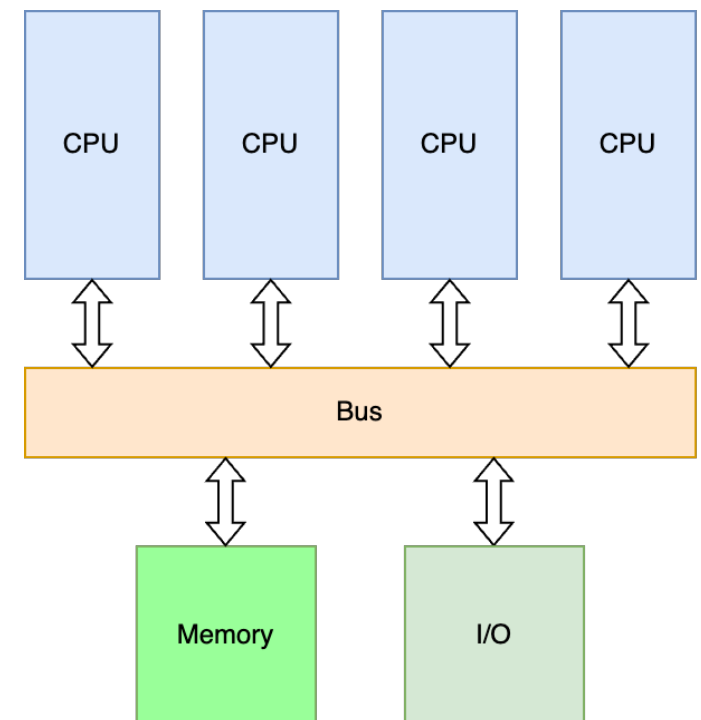
- System bus connects CPU, memory, and I/O
- Master:
 - CPU
- Bottle-neck:
 - single bus for data and instructions
 - memory speed
 - word width
 - at some point: computational power



Traditional Memory Access

Multiple CPU system

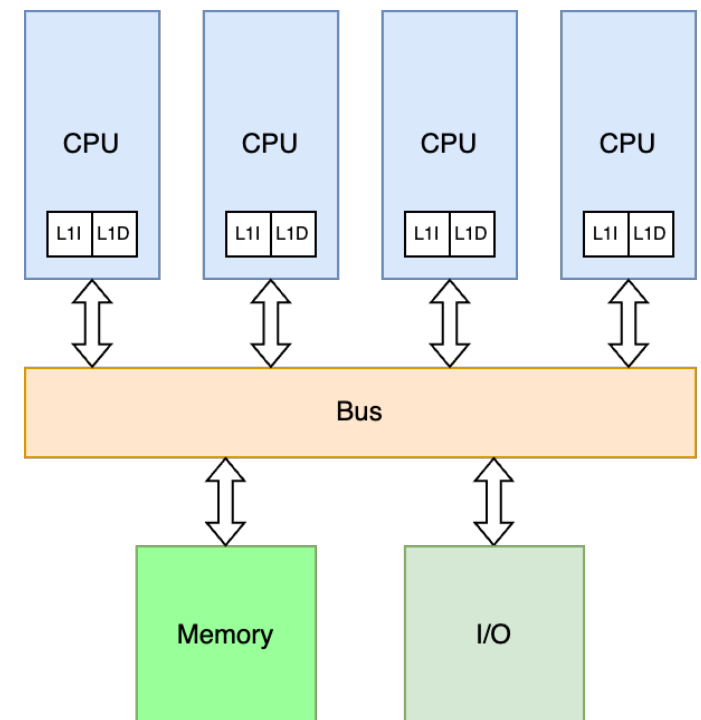
- Multiple CPUs, single memory, I/O
 - more computing performance
- Master: CPUs need to arbitrate system bus access
- Entire memory space available to all CPUs
- Bottle-neck:
 - Single bus, shared → longer bus lanes & arbitration → increased latency
 - Data bandwidth per CPU decreases with more CPUs
- Limitations:
 - number of CPUs can't grow indefinitely
 - CPU stalls while waiting for bus access



Traditional Memory Access, cached

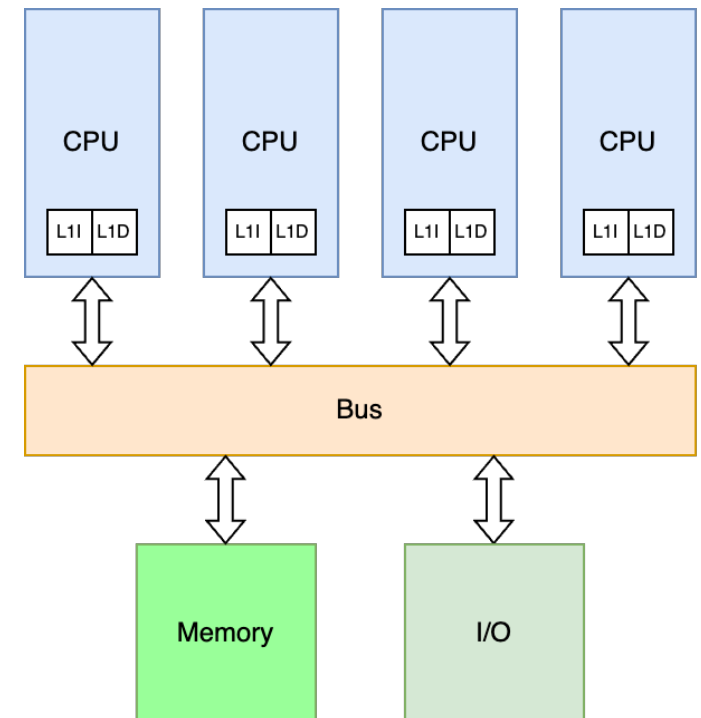
Persisting problems:

- The gap between memory bandwidth and CPU performance is large (still growing today)
- ➔ Introduce **Cache** to bridge the gap
 - brings a multitude of challenges, still active research area
- Cache-coherence problem:
 - multiple copies of a memory block
 - ➔ **snooping**: listen on the bus to what other CPUs are loading from memory
 - ➔ **write invalidation**: writing to memory block invalidates this block in other caches (or immediately replace the cache lines with the updated block)



Traditional Memory Access, Conclusion

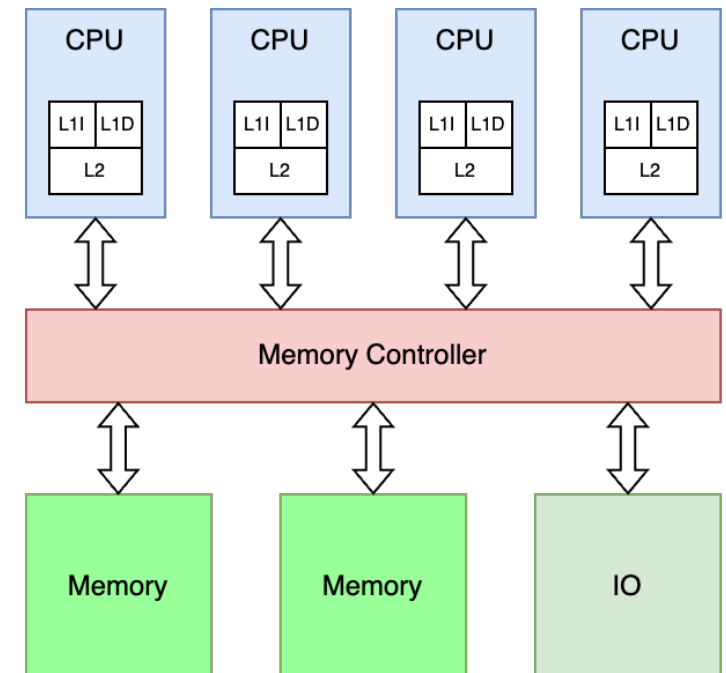
- Memory bandwidth is limited and shared among CPUs
- Bus access needs arbitration. This can introduce non-uniform access times between CPU cores.
- Bus-latency increases with bus-length (signal propagation).
- Cache needed to isolate CPU cores.
- Cache coherence is a challenge.
- Limit of approx. four CPU cores.



Uniform Memory Access Architecture (UMA)

Goal: uniform access time for all CPU cores

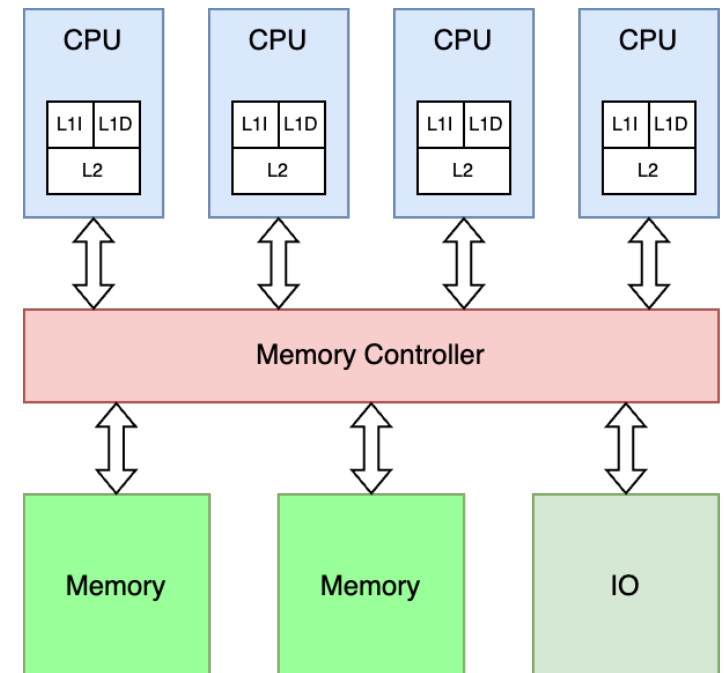
- Replace system bus with something more performant.
- All CPUs have a direct connection to the **Memory Controller** (Northbridge).
- Northbridge needs higher bandwidth than system bus to handle multiple memories and CPUs.
- Additional memory channels possible for increased bandwidth.
- All memories in a single address space.
- L2 cache in each CPU acts as a buffer.



Uniform Memory Access Architecture

Goal: uniform access time for all CPU cores

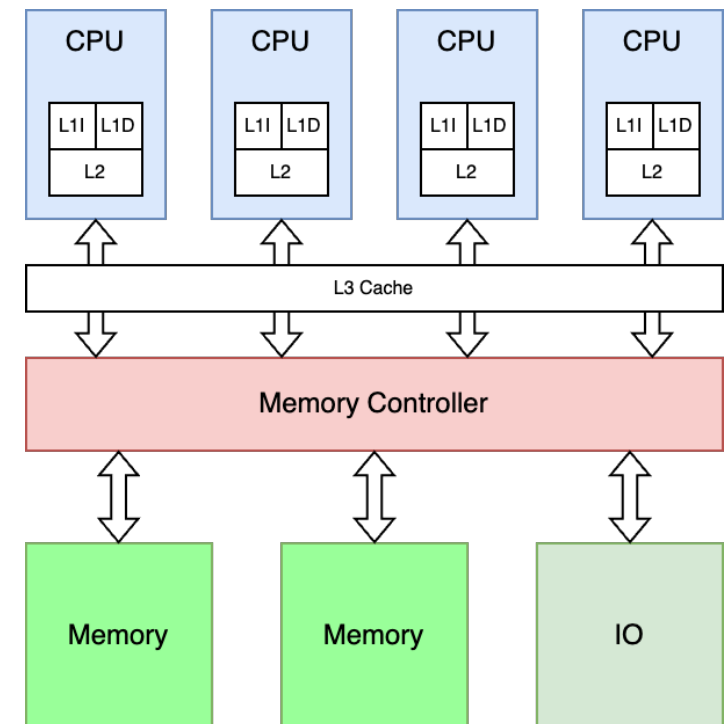
- Uniform path length (access time) from any CPU to any memory
- Cache-coherence remains a challenge
- Limitation:
 - Northbridge bandwidth is limited → does not scale indefinitely!
 - I/O transfers via Northbridge → does not scale!



Uniform Memory Access Architecture

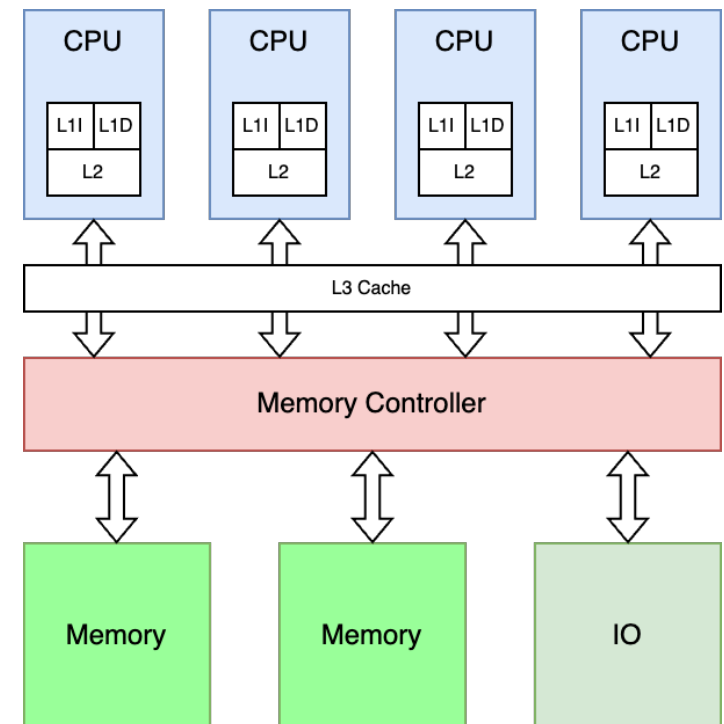
Add another level of cache

- Level 3 cache acts as a buffer between Northbridge and CPUs
- L3 eases cache-coherence problem
 - Snooping happens in L3 cache instead of Northbridge
- Bandwidth limitations remain!
 - ➔ does not scale!



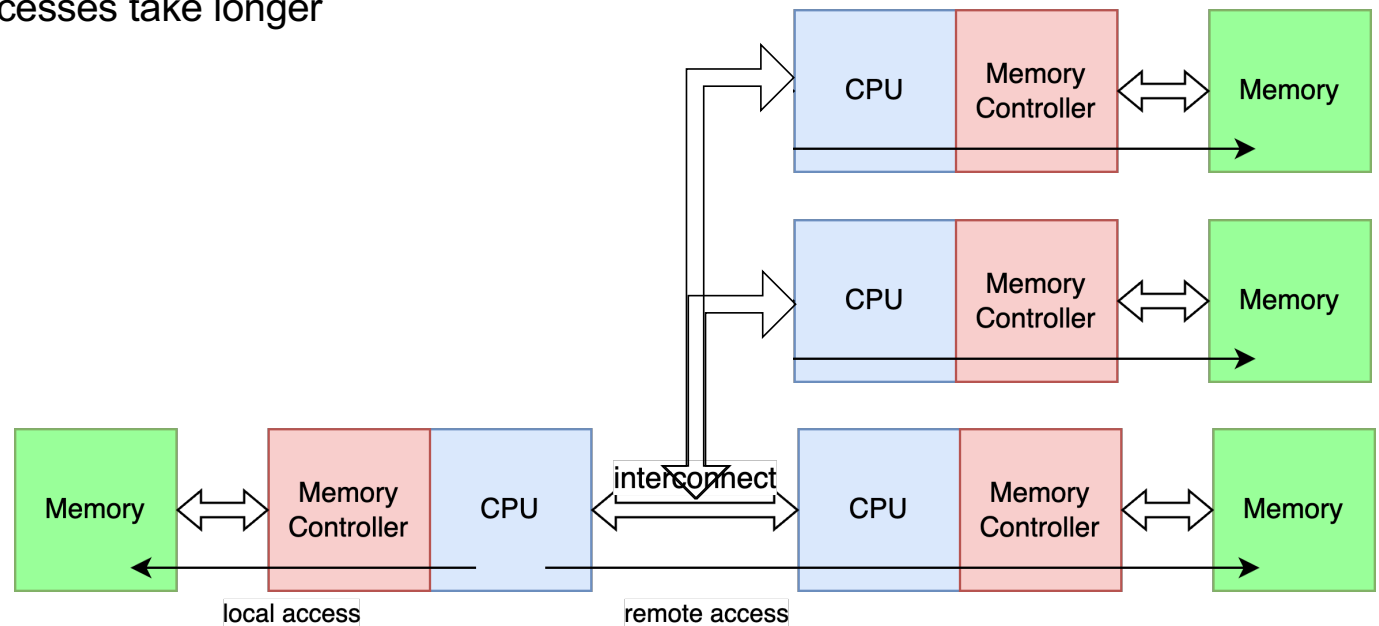
Uniform Memory Access, Conclusion

- Uniform access times for all CPUs
- Local L1/L2 caches in all CPUs
- L3 adds a buffer layer
- Limit of four CPUs still in place



Beyond UMA

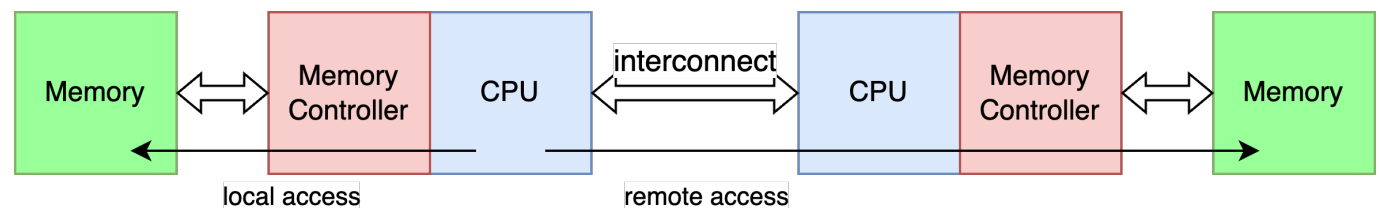
- New approach for larger systems:
Blocks of UMAs could be linked with interconnects
 - adds complexity
 - remote memory accesses take longer



Non-Uniform Memory Access (NUMA)

Local memory for every CPU

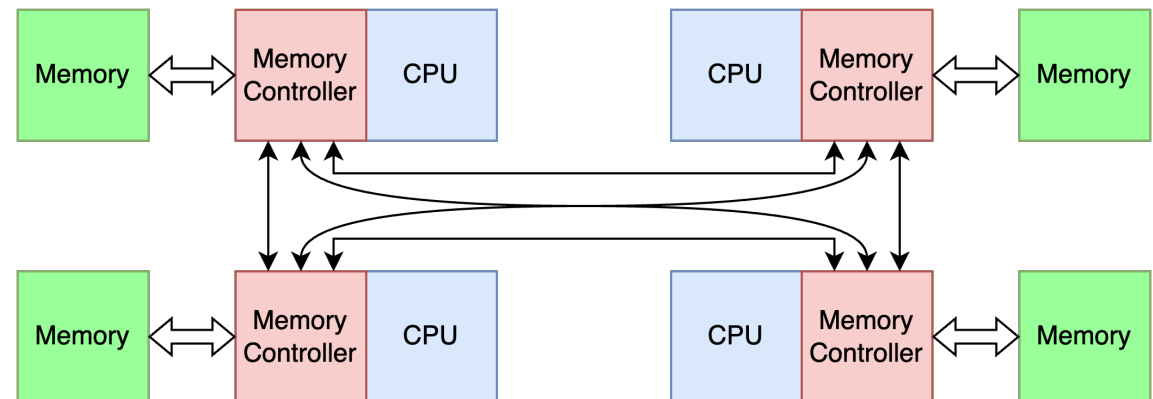
- Embedded has left the building (in most cases)
- Memory is no longer centralized.
- Memory accesses have different latencies for local and remote accesses → non-uniform.
- Each CPU has its own memory address space.
- Remote access via the interconnect and the memory controller in the remote CPU.
- Memory mapping requires software support.
- Cache coherence becomes a lot harder.



Non-Uniform Memory Access (NUMA)

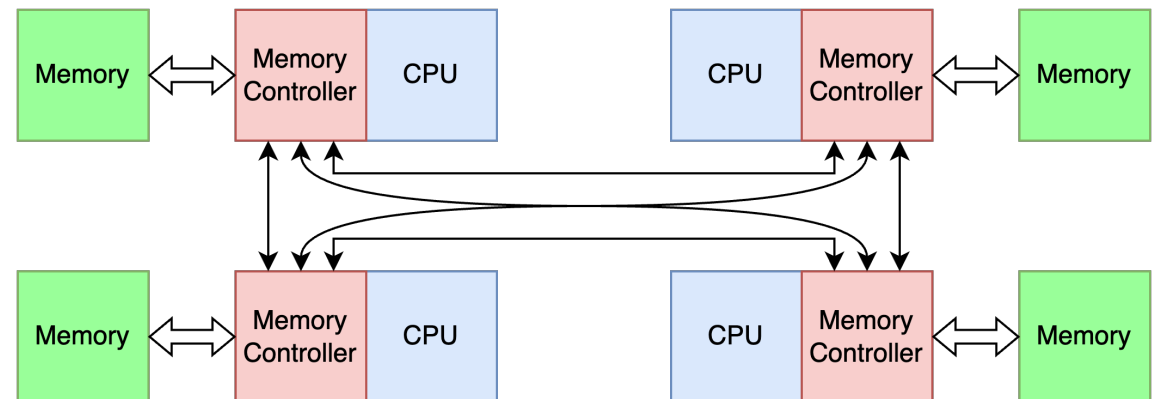
Point-to-point interconnect

- Remote accesses can take multiple hops → stronger non-uniformity.
- Introduce point-to-point interconnect → direct connection to every remote memory.
- Cache-coherence is even more difficult via interconnects.
- You need to find someone to design that logic board or interconnect system.



Non-Uniform Memory Access, Conclusion

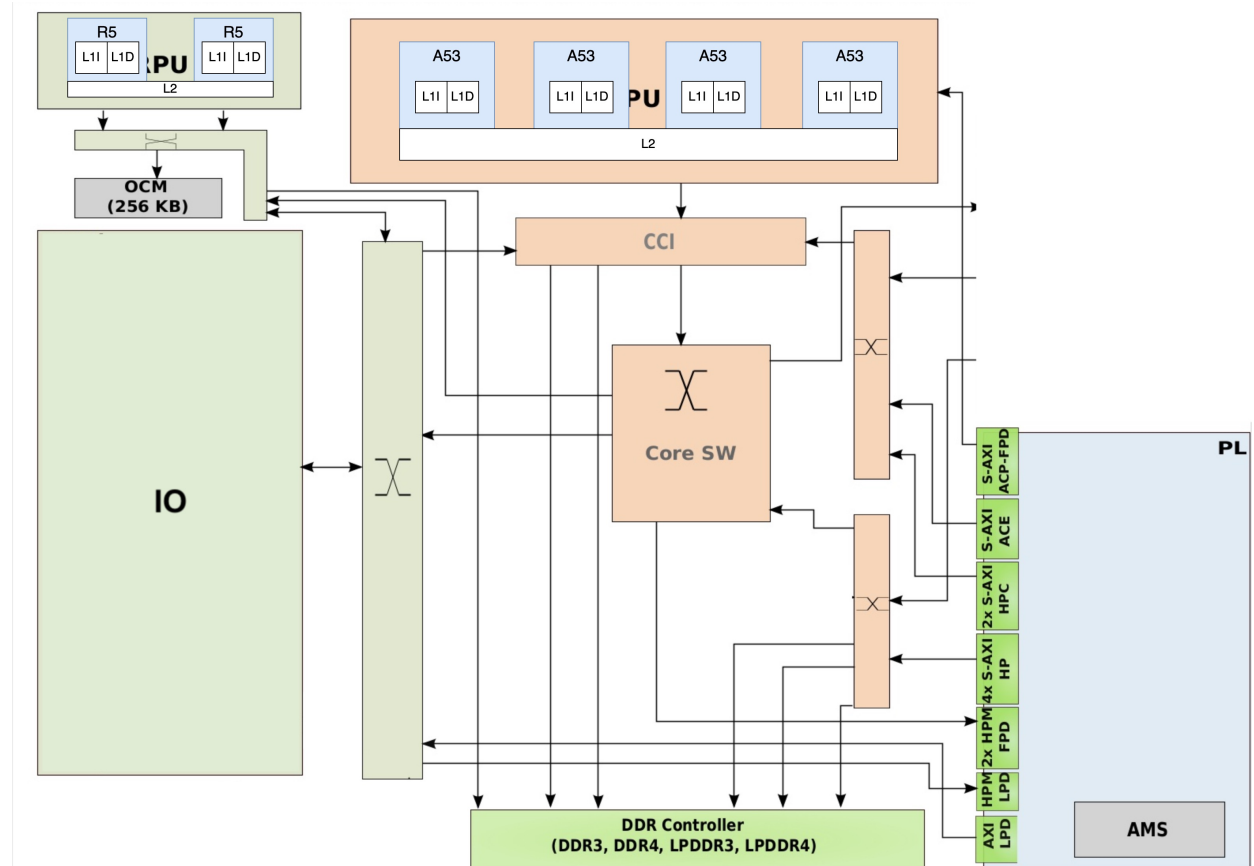
- Local memory in every CPU, fast access times
- Remote memory accessible via interconnects, longer access times
- OS and SW support required for efficient mapping of memory allocation
- Complex cache-coherence



Exercise

Ultra96 Zynq Ultrascale+ EG3 (quad-core A53, dual-core R5)

- UMA? NUMA?



big.LITTLE Architecture

Combine different computing cores in one architecture

- LITTLE cores: energy-efficient, suitable for background tasks and low-intensity workloads.
- big cores: performance-optimized, suitable for high-intensity tasks.
- OS can dynamically allocate tasks to the appropriate core type
- Cores of the same type are organized in clusters

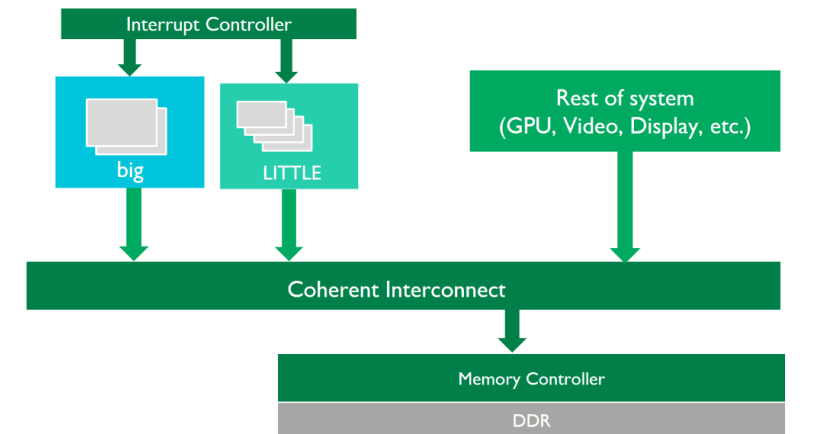


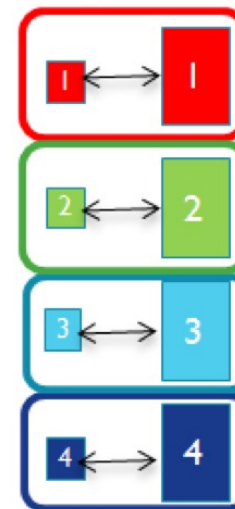
Image: arm.com

big.LITTLE Software Execution

Two major execution models:

- **CPU Migration:**
paired LITTLE and big cores.
 - each pair is visible to the OS as one logical core.
 - Only one core per pair is active at any time.
 - Switch dynamically between big and LITTLE depending on current load.
Uses dynamic voltage and frequency scaling.
 - Simple design vs less flexibility.

CPU Migration



big.LITTLE Software Execution

Two major execution models:

■ Global Task Scheduling

- OS is aware of all big and LITTLE cores and their different capabilities. → visible as individual cores.
- Scheduler can use any subset of available cores simultaneously.
- High-priority/compute-intensive tasks assigned to big cores.
- Low-priority tasks assigned to LITTLE cores.
- More complexity vs higher parallelism and flexibility.

Global Task Scheduling

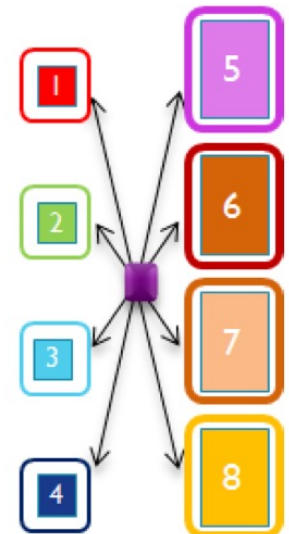


Image: arm.com

big.LITTLE: Global Task Scheduling

Global Task Scheduling Benefits

- The system can have different numbers of big and LITTLE cores.
- Any number of cores may be active.
- All cores can be used simultaneously.
- Example use case:
Assign compute-heavy tasks to big cores (5-8).
Light tasks (UI responsiveness, sensor interrupts) are run on the LITTLE cores only.
➔ Heavy tasks have better completion time, as there are no concurring light tasks.
Light tasks have shorter latency as they don't need to wait for heavy tasks to finish.
- Interrupts can be targeted to big or LITTLE cores, optimizing latency vs compute power.

Global Task Scheduling

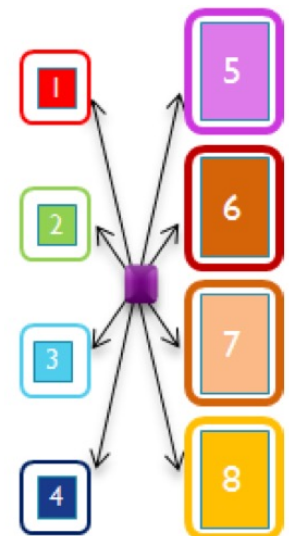
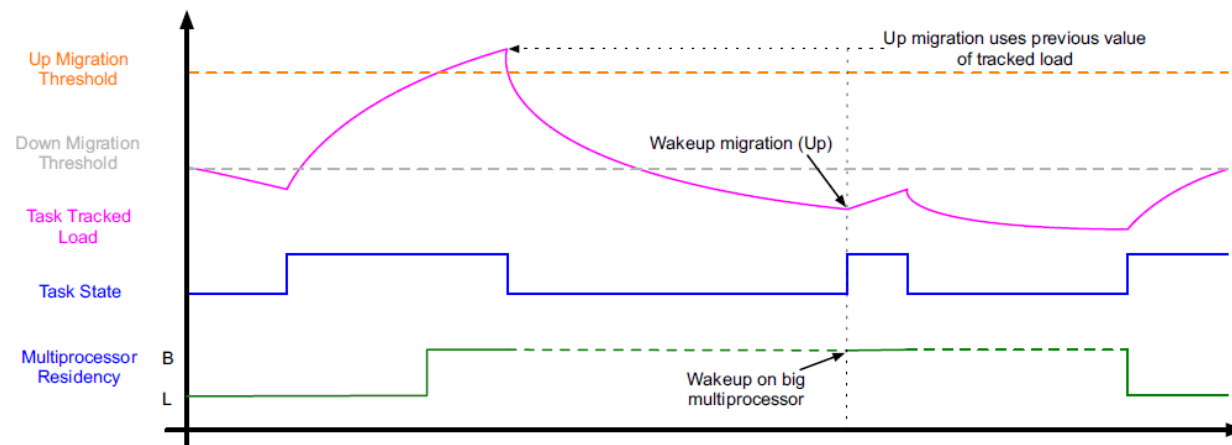


Image: arm.com

big.LITTLE: Global Task Scheduling

Global Task Scheduling

- The compute-heavy tasks can be identified by the scheduler dynamically and in real-time.
- Tracking the load average (moving average) for each task.
- Migrate tasks up (to big core) or down (to LITTLE core) according to load.
- Automatic per-task optimization for power saving/performance optimization.



HS 2025

Global Task Scheduling

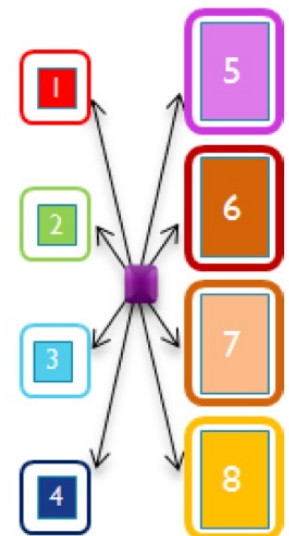


Image: arm.com

big.LITTLE Conclusion

- big.LITTLE architecture enables new software execution models
- CPU migration (pairs of big and little cores)
 - Simpler to handle for the OS
 - Less flexibility
- Global Task scheduling
 - Cores fully visible to OS, provides more flexibility
 - Scheduling gets more complex
- big.LITTLE allows to optimize for power consumption and performance

Summary

- **Traditional Memory Access** does not scale and has a memory bandwidth bottleneck
- **Uniform Memory Access** has the same memory access time for all CPU cores and suffers from similar bottlenecks.
- **Non-Uniform Memory Access** gets rid of the memory bottle neck by giving each CPU local memory. NUMA has varying access times for local and remote accesses.
- **big.LITTLE** architecture is suitable to build flexible systems regarding energy consumption and computing power.